

LETTER

S-COPU: sphere-level speculative scheduling for hardware-accelerated motion planning

Hui Lan¹, Dongliang Xiong^{1, a)}, and Kai Huang¹

Abstract Collision detection remains a critical bottleneck in robotic motion planning. Existing hardware accelerators suffer from *orientation aliasing* and *granularity mismatch* when applied to modern multi-sphere robot models, leading to inefficient link-level predictions. We propose S-COPU, a sphere-level speculative scheduling architecture that aligns prediction granularity with geometric representation. By leveraging parallel spatial hashing and a “generate-then-predict” pipeline, S-COPU reduces expensive geometric queries by approximately 25% and achieves a 14.9% end-to-end latency reduction in high-clutter environments.

Keywords: Motion planning, collision detection, hardware acceleration, spatial hashing, speculative scheduling

Classification: Integrated circuits (memory, logic, analog, RF, sensor)

1. Introduction

Motion planning is a fundamental capability for autonomous robots operating in dynamic environments, enabling tasks ranging from industrial assembly [1] to human-robot collaboration [2]. However, collision detection (CD) remains the primary computational bottleneck, typically consuming over 90% of the total planning time in sampling-based algorithms like RRT* and PRM [3–9]. To address this latency challenge, recent surveys highlight the shift toward domain-specific architectures for robotics and emphasize the critical need for energy-efficient hardware accelerators [10–18].

To ensure safety and enable efficient distance queries, modern motion planning frameworks (e.g., cuRobo [19]) increasingly adopt *sphere-based modeling*, where complex robot links are approximated by **multi-sphere representations** rather than oriented bounding boxes (OBBs) [20, 21]. While prior hardware accelerators like [22] pioneered circuit-based collision checks, and more recent FPGA approaches addressed shortest path problems [23], they primarily focused on simple geometries or graph search phases. To further reduce the computational burden, prediction-based acceleration techniques have emerged, which exploit the spatial locality of obstacles to speculatively skip expensive geometric checks for likely-safe configurations [24–26]. The state-of-the-art collision predictor, COPU [27], utilizes a hardware unit that hashes the *geometric center of a link* into a collision history table (CHT) to look up historical collision states, thereby filtering out redundant queries.

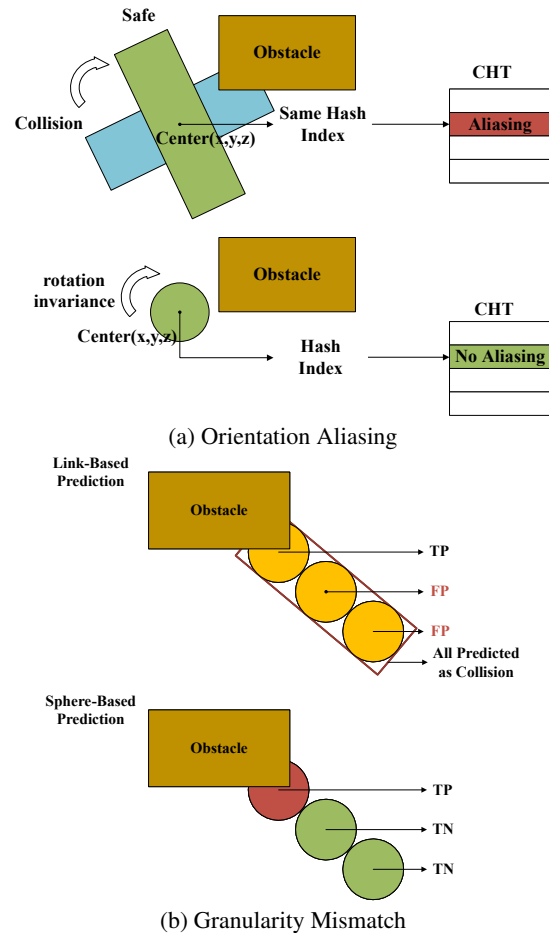


Fig. 1 Illustration of prediction limitations. (a) Orientation aliasing causes ambiguous mappings. (b) Granularity mismatch leads to false positives.

However, this link-based hashing inherently discards orientation information and introduces critical limitations when applied to modern sphere-based models. As illustrated in Fig. 1(a), center-based prediction suffers from **Orientation Aliasing**: distinct “safe” and “colliding” states map to the exact same hash index because the link center remains static during rotation. Furthermore, Fig. 1(b) exposes the **Granularity Mismatch** problem. Since constituent spheres within a single link exhibit divergent collision states, a coarse link-level predictor inevitably flags safe spheres as **False Positives (FP)**, leading to redundant geometric computations. Moreover, it may fail to detect collisions if the link center is predicted as safe while a distal sphere is in collision, resulting in safety-critical **False Negatives (FN)**. S-COPU re-

¹ College of Integrated Circuits, Zhejiang University, Hangzhou, China

^{a)} xiongdl@zju.edu.cn

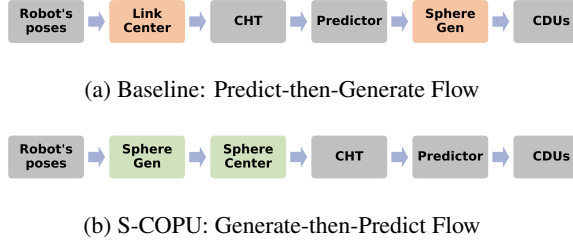


Fig. 2 Comparison of prediction pipelines. (a) Baseline predict-then-generate. (b) S-COPU generate-then-predict.

solves this by correctly identifying **True Negatives (TN)** at the sphere level, enabling precise speculative scheduling of geometric queries.

In this letter, we propose **S-COPU**, a **Sphere-Level Speculative Scheduling** architecture to resolve these limitations. Unlike prior work that predicts collision at the coarse link level, our approach performs fine-grained prediction based on the exact spatial coordinates of individual spheres. We demonstrate that by aligning the prediction granularity with the detection granularity, we can significantly improve prediction recall and precision. Crucially, by leveraging the *spatial locality inherent in hashing*, our method accommodates the increased number of prediction points without expanding the on-chip memory size.

The main contributions of this work are summarized as follows:

- We identify the orientation aliasing and granularity mismatch problems in existing collision prediction accelerators when handling sphere-based robot models.
- We propose a hardware architecture that integrates sphere-level hashing and speculative query scheduling, leveraging fine-grained prediction to achieve a significant reduction in expensive geometric computations.
- Experimental results on a 7-DOF robot show that our approach reduces collision detection queries (CDQs) by approximately 25% compared to the state-of-the-art link-level predictor [27] in high-clutter environments.

2. Proposed method

2.1 System overview

The baseline architecture, COPU [27], adopts a “predict-then-generate” pipeline to minimize computational overhead. As shown in Fig. 2(a), the hardware first hashes the geometric center of a robot link to query the CHT. Only when a collision is predicted does the system generate the detailed bounding volumes for intersection testing. While effective for single-body links, it creates a critical blind spot for **sphere-based models** widely used in modern planners [19]. By discarding orientation information (*orientation aliasing*) and failing to capture the spatial distribution of spheres (*granularity mismatch*), the baseline predictor suffers from low recall.

To address this, we propose **S-COPU (sphere-based COPU)**, which restructures the pipeline into a “generate-then-predict” flow (Fig. 2(b)). Unlike the baseline, S-COPU moves the *Sphere Generation Unit* to the front. This inversion allows access to exact sphere coordinates *before*

Algorithm 1 Sphere-Level Speculative Collision Detection

Input: Robot Configuration q , CHT, Threshold S

Output: Collision Status (True/False)

```

1: // Phase 1: Sphere Generation (Pre-computation)
2:  $Spheres \leftarrow \text{ForwardKinematics}(q)$ 
3: // Phase 2: Parallel Prediction
4: for all  $s \in Spheres$  do ▶ Executed in parallel by  $P$  predictors
5:    $idx \leftarrow \text{Hash}(s.center)$ 
6:    $(C_{coll}, C_{noncoll}) \leftarrow CHT[idx]$ 
7:    $Q \leftarrow (C_{coll} > C_{noncoll} \cdot S) ? Q_{COLL} : Q_{NONCOLL}$ 
8:   Push  $s$  to  $Q$ 
9: end for
10: // Phase 3: Speculative Execution (CDU)
11: while  $Q_{COLL} \neq \emptyset \vee Q_{NONCOLL} \neq \emptyset$  do
12:    $s_{target} \leftarrow \text{Pop}(Q_{COLL} \neq \emptyset ? Q_{COLL} : Q_{NONCOLL})$ 
13:    $IsColliding \leftarrow \text{GeometricCheck}(s_{target}, Environment)$ 
14:    $\text{UpdateCHT}(s_{target}, IsColliding)$ 
15:   if  $IsColliding$  is True then
16:     return True
17:   end if
18: end while
19: return False

```

scheduling, bypassing link-center orientation ambiguity. By aligning prediction granularity with detection, S-COPU enables speculative scheduling of individual spheres.

2.2 Sphere-level prediction algorithm

To resolve these limitations, we adopt a sphere-centric speculative strategy. The detailed execution flow is formally defined in **Algorithm 1**.

Algorithm Overview: The workflow in Algorithm 1 inverts the conventional pipeline. Instead of relying on the link center, we first generate exact world-frame coordinates for all constituent spheres (Phase 1). In Phase 2, we adapt the coordinate hashing [27] to the sphere level by using the **most significant bits (MSBs)** of each sphere’s center. This mapping ensures that the prediction index directly reflects the spatial occupancy of the geometry, enabling the dispatcher to sort spheres into priority queues ($Q_{COLL}/Q_{NONCOLL}$) based on their individual collision risks (Phase 3).

Constant Memory Footprint: Critically, tracking significantly more points ($N_{spheres} \gg N_{links}$) does not necessitate a larger CHT. The memory footprint is determined strictly by the **spatial address range** (defined by the workspace volume and discretization precision Δ) and the entry bit-width, regardless of the number of query points. As confirmed in Section 4., S-COPU maintains high accuracy within the baseline’s 4KB budget, leveraging the fixed address space to accommodate fine-grained queries without resource expansion.

2.3 Parallel hardware architecture

To implement the proposed sphere-level speculative scheduling, we redesigned the hardware pipeline as shown in Fig. 3. Compared to the baseline architecture [27], S-COPU introduces two critical architectural modifications:

1) Pipeline Reordering (Early Sphere Generation): In the baseline architecture, geometric primitives are generated lazily only after a collision is predicted or during the detection phase. In contrast, S-COPU restructures the pipeline

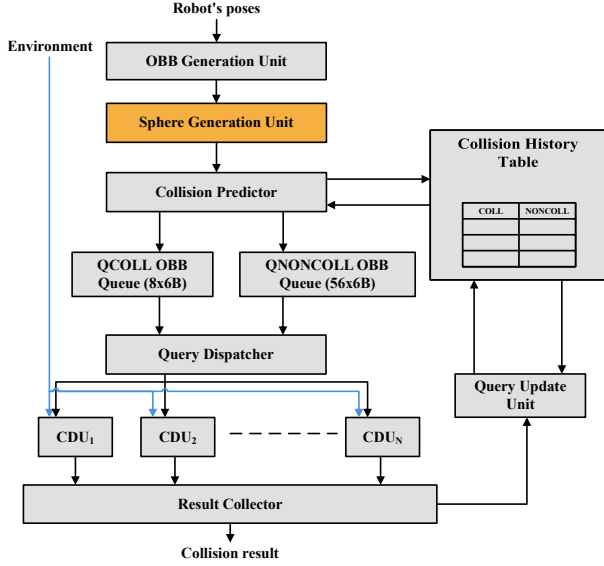


Fig. 3 Hardware microarchitecture of S-COPU featuring upstream sphere generation and vectorized collision predictors.

by relocating the **Sphere Generation Unit** to immediately follow the Forward Kinematics Engine. This architectural shift ensures that the world-frame coordinates of all constituent spheres are computed *before* the prediction stage. This “Early Generation” strategy provides the Collision Predictor with the precise spatial data required for the fine-grained hashing (Algorithm 1), enabling the system to filter tasks at the sphere level rather than the coarse link level.

2) Vectorized Collision Predictors: The transition from link-level to sphere-level prediction increases the query frequency by a factor of K (spheres per link). To prevent the prediction stage from becoming a bottleneck, we parallelize the prediction logic. S-COPU deploys a bank of P **Parallel Collision Predictors** ($P = 4$ in our implementation, empirically matched to the sphere generation throughput). Each predictor operates independently to process the stream of sphere coordinates, executing the hashing and table lookup in parallel. Since these predictors rely exclusively on lightweight integer arithmetic (as discussed in Section 4.5), this 4-way parallelization incurs negligible area overhead while ensuring that the prediction throughput matches the burst rate of the upstream generation units.

3. Theoretical cost analysis

In this section, we formulate a mathematical model to quantify the expected computational cost of collision detection. This analysis aims to establish a theoretical upper bound for the efficiency gains achievable by speculative scheduling.

3.1 Problem formulation and assumptions

We model the collision detection phase as a global search problem over the total set of N geometric primitives (spheres), denoted as $\mathcal{S} = \{s_1, s_2, \dots, s_N\}$. For the 7-DOF KUKA LBR iiwa robot used in this work, the model comprises $N = 22$ spheres distributed across 7 links. The geometric check phase follows an **Early Exit** policy: the process terminates immediately once the first colliding sphere

is identified.

To make the derivation analytically tractable, we introduce the following statistical assumptions:

- **Assumption 1 (Bernoulli Trials):** The collision state of each sphere is modeled as a Bernoulli random variable. Let $X_i \in \{0, 1\}$ denote the state of sphere s_i , where $X_i = 1$ indicates a collision.
- **Assumption 2 (Independence):** We assume the collision events are independent and identically distributed (i.i.d.). That is, $P(X_i = 1|X_j = 1) = P(X_i = 1) = P$, where P is the prior collision probability.

While real-world obstacles exhibit spatial correlation (violating Assumption 2), this idealized model provides a necessary baseline for evaluating algorithmic efficiency under maximum entropy conditions.

3.2 Expectation of geometric queries

We define the cost K as the number of geometric queries executed before termination.

1) Baseline (Random Scheduling): Without prediction, spheres are checked in a random order. The probability that the process terminates exactly at the k -th step is $P(1-P)^{k-1}$. Summing over all possibilities, the expected cost E_{base} is governed by the geometric distribution:

$$E_{base} = \sum_{k=1}^N k \cdot P(1-P)^{k-1} = \frac{1 - (1-P)^N}{P} \quad (1)$$

2) Speculative Scheduling (S-COPU): Our method reorders the sequence based on a predictor with Precision A and Recall C . The set $\mathcal{S}$ is partitioned into a high-priority queue Q_{coll} (predicted positive) and a low-priority queue $Q_{noncoll}$ (predicted negative). By applying the Law of Total Expectation on the size of Q_{coll} , we derive the expected cost E_{pred} as:

$$E_{pred} = \frac{1 - (1-CP)^N}{A} + \frac{(1-CP)^N - (1-P)^N}{P_{rem}} \quad (2)$$

where $P_{rem} = \frac{(1-C)P}{1-CP/A}$ is the conditional collision probability for spheres in $Q_{noncoll}$.

3.3 Model validation

To verify Eq. 2, we conducted Monte Carlo simulations (10^4 trials) under strict i.i.d. conditions. As shown in Fig. 4, simulation results align strictly with the theoretical curve, with a maximum relative error $<1.5\%$. This confirms the mathematical rigor of our derivation within the defined scope.

4. Experimental results

4.1 Methodology

Benchmarks: We evaluate the proposed architecture using a 7-DOF KUKA LBR iiwa 7 robotic arm, a standard platform in sampling-based planning research [5]. To strictly align with state-of-the-art collision checking standards, we adopt the optimized **sphere-based representation** provided by the cuRobo library [19]. In this model, the geometry of each kinematic link is approximated by a set of spheres (averaging 3 spheres per link), resulting in a $3\times$ increase in

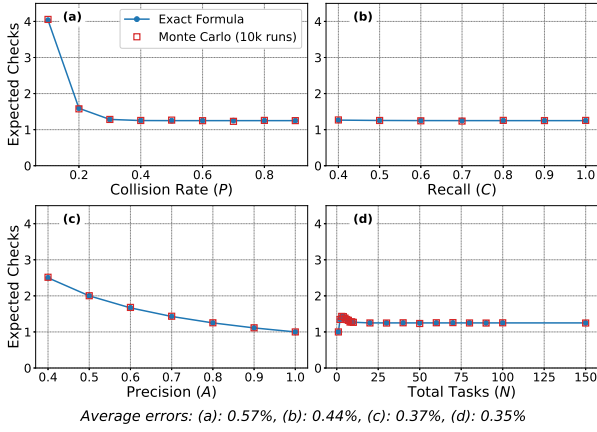


Fig. 4 Validation of the theoretical model against Monte Carlo simulations. Subplots (a)–(d) show the impact of varying probability P , recall C , precision A , and sphere count N , respectively.

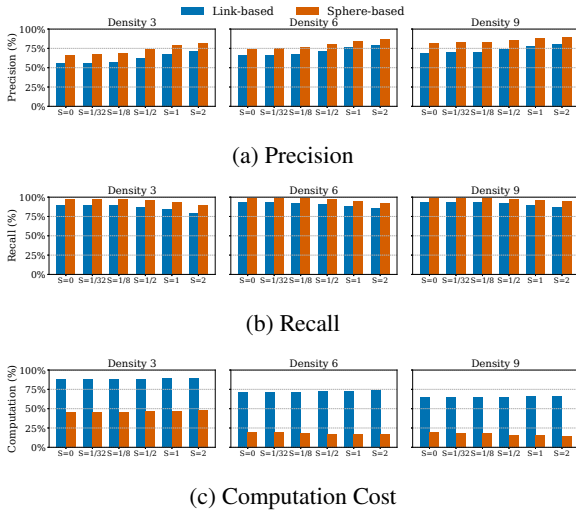


Fig. 5 Comparison of prediction metrics (Precision, Recall, Cost) between link-based and sphere-based predictors across varying obstacle densities.

tracking points compared to the baseline link-level predictor. We generate 1,000 random planning problems across environments with varying clutter levels [28], categorized into five difficulty groups (G1–G5), ranging from sparse to high-clutter.

Baselines: We compare four configurations: 1) **Baseline:** A standard accelerator without collision prediction; 2) **COPU (Prior Work)** [27]: The state-of-the-art predictor using link-center hashing; 3) **S-COPU (Proposed):** Our sphere-level predictor; and 4) **Oracle:** An ideal predictor with perfect knowledge.

Evaluation Platform: We developed a custom cycle-accurate architectural simulator (CHT size constrained to 4KB).

4.2 Evaluation of prediction quality and design aspects

We evaluate the prediction performance and design sensitivity of S-COPU compared to the link-level baseline across varying obstacle densities and prediction thresholds (S).

1) Metrics and Density Scaling: Fig. 5 presents a comprehensive sweep of metrics. As obstacle density increases,

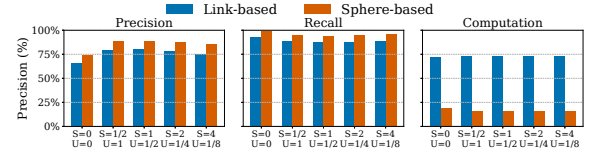


Fig. 6 Impact of update frequency (U) and threshold (S) on prediction performance (Recall, Cost, and Precision).

the link-based predictor fails to filter false positives effectively, maintaining a high Computational Cost compared to S-COPU (subplot (c)). Specifically, at Density 9 with $S = 1$, the link-based method incurs a high computational cost of 65.6%, whereas S-COPU drastically reduces this to 15.3%. Furthermore, while the link-based recall drops to 90.3%, S-COPU maintains a robust safety margin with 96.5% recall. This confirms that sphere-level granularity is essential for robust scaling in cluttered environments.

2) Prediction Threshold (S): The threshold S regulates the aggressiveness of the speculative scheduler. As shown in Fig. 5, increasing S (more conservative) improves Precision but slightly reduces Recall. For instance, at Density 9, S-COPU Precision improves from 82.1% ($S = 0$) to 90.1% ($S = 2$). To balance safety and efficiency, we select $S = 1$ as the operating point. At $S = 1$, S-COPU achieves a favorable trade-off: it retains high recall ($>96\%$) to ensure valid collision alerts while keeping the computational cost significantly lower than the baseline.

3) Update Frequency (U): We further investigate the impact of the CHT update frequency U to optimize bandwidth usage. Fig. 6 illustrates the performance trade-offs for various S and U combinations. Ideally, we aim to minimize U to save SRAM bandwidth. The results indicate that S-COPU is highly robust to reduced updates. When lowering the update frequency from $U = 1/2$ (at $S = 1$) to $U = 1/8$ (at $S = 4$), the Computational Cost remains stable. This validates that S-COPU can effectively operate in a bandwidth-constrained regime ($U = 1/8$) without compromising safety or performance.

4.3 Acceleration for motion planning

Next, we assess the impact of these prediction gains on the overall motion planning pipeline, measuring the reduction in geometric queries and the final end-to-end latency in realistic scenarios.

1) CDQ Workload Reduction: We measure Collision Detection Queries (CDQs) across five difficulty levels (G1–G5). As shown in Fig. 7(a), in the most demanding scenario (G5), the link-level predictor requires 38.5% of the baseline workload. S-COPU reduces this to 28.9%. This constitutes a $\sim 25\%$ reduction over COPU, effectively bridging 66% of the gap to the Oracle limit (24.0%).

2) End-to-End Speedup: Fig. 7(b) compares the total execution cycles, accounting for both prediction overhead and geometric check savings. S-COPU consistently achieves lower latency than the baseline. Notably, the performance advantage **widens** as the environment becomes more complex, increasing from 8.0% in sparse scenes (G1) to 14.9% in high-clutter scenarios (G5). This latency reduction cor-

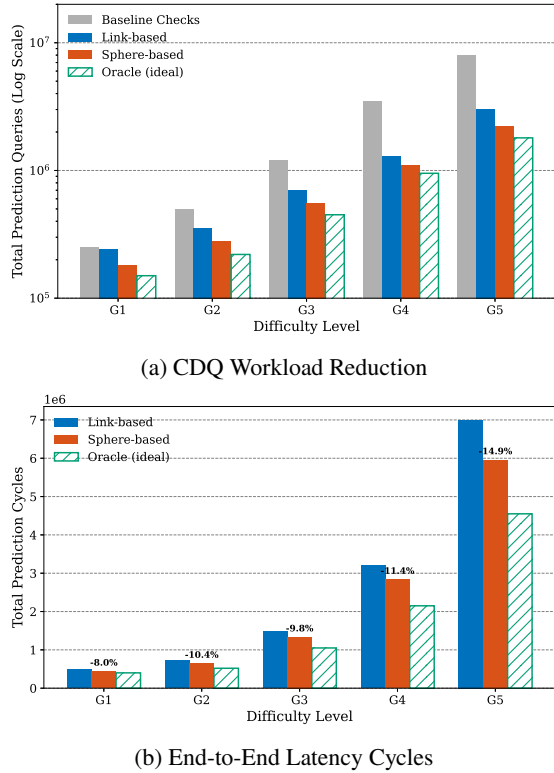


Fig. 7 Motion planning acceleration results. (a) Reduction in collision detection queries (CDQs). (b) End-to-end latency comparison showing speedup.

responds to a net speedup of approximately **1.18×** over the link-level baseline. Although fine-grained prediction introduces a marginal increase in hash computations, these operations are computationally cheap compared to the heavy arithmetic required for geometric intersection tests.

3) Resource Efficiency: Crucially, S-COPU achieves this performance gain without increasing the on-chip memory requirement (fixed at 4KB). This confirms that our spatial hashing scheme effectively compresses the increased number of sphere coordinates into the same storage footprint, making it a highly area-efficient solution.

4.4 Discrepancy analysis

Comparing the algorithmic scaling in Section 4.2 with the system-level acceleration in Section 4.3 reveals a distinct performance divergence. As observed in Section 4.2, the sphere-based strategy reduces the computational cost to roughly a quarter of the link-based baseline (e.g., 15.3% vs. 65.6% at Density 9). However, the realized reduction in **actual planning workload** falls short of the theoretical upper bound derived in Section 3.. We attribute this discrepancy to three intrinsic characteristics of real-world planning that violate the ideal Bernoulli assumptions: 1) *Sampling Bias in Boundary Regions*: Motion planners actively explore narrow passages near obstacle boundaries where spatial discretization leads to ambiguous hash mappings. 2) *Cost Asymmetry of Safe Queries*: In safe trajectories, all potential collisions (including False Positives) must be verified, diluting the gain from prediction. 3) *Spatial Correlation*: Rigid-body constraints cause false positives to appear in clusters rather than as isolated events, reducing the effective

entropy of the query sequence.

4.5 Hardware overhead analysis

Although a full RTL implementation is outside the scope of this letter, we estimate overhead by projecting synthesis data from baseline implementations [4, 27] (TSMC 45nm [29, 30]).

1) Area Overhead (Negligible): According to the synthesis results in [4], the hardware area of a collision detection accelerator is dominated by the Intersection Units within the CDUs (approx. $0.25mm^2$ per unit). In comparison, the baseline prediction logic incurs an area overhead of less than 2% [27]. In S-COPU, additional area stems from the **Sphere Generation Unit** and vectorized **Collision Predictors** ($P = 4$). Since sphere generation uses basic vector transformations (unlike OBB matrix ops) and predictors use bitwise logic, their area is negligible compared to arithmetic-intensive CDUs. Consequently, we estimate the aggregate logic area overhead to remain **below 3%**, while inheriting the identical 4KB CHT memory.

2) Power Efficiency (Positive Trade-off): We analyze power efficiency by contrasting the energy cost per operation type. Geometric intersection tests are power-hungry ($> 30mW$ per pipelined unit [4]) due to heavy arithmetic logic. Conversely, the collision prediction logic accounts for only $\approx 1\%$ of the total energy consumption in the baseline [27]. Although S-COPU increases the active duty cycle of the prediction stage (processing K spheres per link with $P = 4$ parallel units), this increase involves primarily low-energy integer switching. This overhead is vastly outweighed by the energy savings from the **25% reduction in CDQs** (Section 4.3), effectively trading cheap integer operations to prevent expensive geometric computations.

5. Conclusion

In this letter, we address the critical *orientation aliasing* and *granularity mismatch* in existing hardware collision predictors. We demonstrate that while link-level prediction suits bounding boxes, it fails to capture the spatial nuances of multi-sphere models, leading to excessive false positives and safety-critical false negatives.

To bridge this gap, we proposed **S-COPU**, a sphere-level speculative scheduling architecture. By inverting the pipeline to a “generate-then-predict” flow and leveraging parallel spatial hashing, S-COPU aligns the prediction granularity with the underlying geometric representation. Our architectural simulation results show that S-COPU reduces the volume of expensive collision detection queries (CDQs) by approximately **25%** and achieves a **14.9%** reduction in end-to-end latency in high-clutter scenarios compared to the state-of-the-art link-level baseline. Furthermore, we demonstrated that by exploiting the spatial locality inherent in the hashing scheme, this fine-grained precision is achieved with only a marginal logic area overhead and *zero* increase in on-chip memory requirements. This work establishes fine-grained speculation as a scalable pathway for enabling real-time, safe motion planning for complex next-generation robots.

References

- [1] M. Onishi, T. Hasegawa, *et al.*, “Motion planning of bimanual robot using adaptive model of assembly,” *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol.E91-A, no.12, 2008.
- [2] Y. Zhao *et al.*, “SPSD: Semantics and deep reinforcement learning based motion planning for supermarket robot,” *IEICE Transactions on Information and Systems*, vol.E106.D, no.5, pp.765–772, 2023.
- [3] J. Pan and D. Manocha, “Gpu-based parallel collision detection for fast motion planning,” *The International Journal of Robotics Research*, vol.31, no.2, pp.187–200, 2012.
- [4] D. Shah, N. Yang, and T.M. Aamodt, “Energy-efficient realtime motion planning,” *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA)*, pp.1098–1114, IEEE, 2023.
- [5] S. Karaman and E. Frazzoli, “Sampling-based algorithms for optimal motion planning,” *The International Journal of Robotics Research*, vol.30, no.7, pp.846–894, 2011.
- [6] J.D. Gammell, T.D. Barfoot, and S.S. Srinivasa, “Batch informed trees (BIT*): Informed asymptotically optimal anytime search,” *The International Journal of Robotics Research*, vol.39, no.5, pp.543–567, 2020.
- [7] A.H. Qureshi, Y. Miao, A. Simeonov, and M.C. Yip, “Motion planning networks: Bridging the gap between learning-based and classical motion planners,” *IEEE Transactions on Robotics*, vol.37, no.1, pp.48–66, 2021.
- [8] J.D. Gammell, S.S. Srinivasa, and T.D. Barfoot, “Informed rrt*: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic,” *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp.2997–3004, IEEE, 2014.
- [9] C. Ericson, *Real-time collision detection*, CRC Press, 2004.
- [10] H. Xu, Y. Yang, F. Min, J. Huang, X. Chen, Y. Han, and N. Sun, “Toward efficient computing for robotics: From a circuit and system view,” *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol.69, no.7, pp.3051–3055, 2022.
- [11] Z. Wan, A. Lele, B. Yu, S. Liu, Y. Wang, V.J. Reddi, C. Hao, and A. Raychowdhury, “Robotic computing on fpgas: Current progress, research challenges, and opportunities,” *Proceedings of the 2022 IEEE 4th International Conference on Artificial Intelligence Circuits and Systems (AICAS)*, pp.291–295, IEEE, 2022.
- [12] S. Murray, W. Floyd-Jones, Y. Qi, G. Konidaris, and D.J. Sorin, “The microarchitecture of a real-time robot motion planning accelerator,” *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp.1–12, IEEE, 2016.
- [13] Y. Han, Y. Yang, X. Chen, and S. Lian, “Dadu series: Fast and efficient robot accelerators,” *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, pp.1–8, IEEE, 2020.
- [14] S. Lian, Y. Han, X. Chen, Y. Wang, and H. Xiao, “Dadu-p: A scalable accelerator for robot motion planning in a dynamic environment,” *Proceedings of the 55th Annual Design Automation Conference (DAC)*, pp.1–6, ACM, 2018.
- [15] Y. Yang, X. Chen, and Y. Han, “Dadu-cd: Fast and efficient processing-in-memory accelerator for collision detection,” *2020 57th ACM/IEEE Design Automation Conference (DAC)*, pp.1–6, IEEE, 2020.
- [16] M. Bakhshalipour, S.B. Ehsani, M. Qadri, D. Guri, M. Likhachev, and P.B. Gibbons, “Racod: Algorithm/hardware co-design for mobile robot path planning,” *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA)*, pp.1030–1045, ACM, 2022.
- [17] T. Jia, E.Y. Yang, Y.S. Hsiao, J. Cruz, D. Brooks, G.Y. Wei, and V.J. Reddi, “Omu: A probabilistic 3d occupancy mapping accelerator for real-time octomap at the edge,” *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp.909–914, IEEE, 2022.
- [18] T.Y. Yeh, P. Faloutsos, and G.D. Reinman, “Accelerating real-time physics simulation by leveraging high-level information,” *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, ACM, 2021.
- [19] B. Sundaralingam, S.K.S. Hari, A. Fishman, C. Garrett, K. Van Wyk, V. Blukis, A. Millane, H. Oleynikova, A. Handa, F. Ramos, *et al.*, “curobo: Parallelized collision-free robot motion generation,” *Proceedings of the 2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp.8112–8119, IEEE, 2023.
- [20] J.W. Chang, W. Wang, and M.S. Kim, “Efficient collision detection using a dual obb-sphere bounding volume hierarchy,” *Computer-Aided Design*, vol.42, no.1, pp.50–57, 2010.
- [21] J. Klosowski, M. Held, J.S. Mitchell, H. Sowizral, and K. Zikan, “Efficient collision detection using bounding volume hierarchies of k-dops,” *IEEE Transactions on Visualization and Computer Graphics*, vol.4, no.1, pp.21–36, 1998.
- [22] S. Murray, W. Floyd-Jones, Y. Qi, D.J. Sorin, and G. Konidaris, “Robot motion planning on a chip,” *Proceedings of Robotics: Science and Systems (RSS)*, 2016.
- [23] L.T.C. Esteves, W.L.A. de Oliveira, and P.C.M.d.A. Farias, “Analysis and construction of hardware accelerators for calculating the shortest path in real-time robot route planning,” *Electronics*, vol.13, no.11, p.2167, 2024.
- [24] C. Yu and S. Gao, “Reducing collision checking for sampling-based motion planning using graph neural networks,” *Proceedings of the 35th International Conference on Neural Information Processing Systems (NeurIPS)*, 2021.
- [25] M. Saha *et al.*, “Speculative computation in robotics: A survey of hardware acceleration techniques,” *Journal of Real-Time Image Processing*, vol.21, p.112, 2024.
- [26] X. Zhang *et al.*, “Hardware-efficient spatial hashing for real-time 3d collision queries,” *IEEE Solid-State Circuits Letters*, vol.7, pp.45–48, 2024.
- [27] D. Shah and T.M. Aamodt, “Collision prediction for robotics accelerators,” *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*, pp.395–410, IEEE, 2024.
- [28] E. Coumans and Y. Bai, “Pybullet, a python module for physics simulation for games, robotics and machine learning.” <http://pybullet.org>, 2016–2021.
- [29] J.E. Stine, L.D. Castellanos, M.H. Wood, J. Henson, F. Love, W.R. Davis, P.D. Franzon, M. Bucher, S. Basavarajaiah, J. Oh, and R. Jenkal, “Freepdk: An open-source variation-aware design kit,” *2007 IEEE International Conference on Microelectronic Systems Education (MSE)*, pp.173–174, IEEE, 2007.
- [30] M.R. Guthaus, J.E. Stine, S. Ataei, B. Chen, B. Wu, and M. Sarwar, “Openram: An open-source memory compiler,” *2016 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pp.1–6, IEEE, 2016.